DATA ENGINEERING • MODULE 03

ETL & Data Pipelines

Move and Transform Data Reliably at Scale

■ PIPELINES

■ LEARNING OUTCOME

By the end of this module you will design, build, test, and monitor production ETL/ELT pipelines in Python, implement data quality checks with Great Expectations, and handle failures gracefully.

■ Skills You Will Gain

- Design ETL vs ELT pipelines with correct patterns
- Build a Python ETL pipeline with extract, transform, load functions
- Implement incremental vs full-load strategies
- Apply data quality checks with Great Expectations
- Handle pipeline failures with retries, alerting, and dead-letter patterns
- Write idempotent pipelines safe to run multiple times

■ **Bad pipelines create bad data, which drives bad decisions. A DE who builds reliable, tested, monitored pipelines is worth 3x a DE who just moves data.**

SECTION 1

Pipeline Architecture Patterns

Pattern	Description	Best For
Full Load	Extract ALL data every run — simple but slow for large tables	Small reference tables, static data
Incremental Load	Extract only rows updated since last watermark timestamp	Large tables with updated_at column
CDC (Change Data Capture)	Capture every INSERT/UPDATE/DELETE from DB transaction log	Real-time, full audit trail
Snapshot	Store complete copy of table per day/week for historical comparison	Slowly changing reference data
Lambda Architecture	Batch layer + Speed layer running in parallel	Low latency + historical completeness
Kappa Architecture	Streaming only — reprocess from Kafka log when needed	Simpler than Lambda, Kafka-native

SECTION 2

Python ETL Pipeline

```
import pandas as pd import sqlalchemy from datetime import datetime, timedelta import logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s') logger =
logging.getLogger(__name__) # EXTRACT — pull only new rows using watermark def extract(last_run_ts:
str) -> pd.DataFrame: engine = sqlalchemy.create_engine('postgresql://user:pass@host/db') df =
pd.read_sql( f"SELECT * FROM orders WHERE updated_at > '{last_run_ts}'", engine )
logger.info(f'Extracted {len(df)} rows since {last_run_ts}') return df # TRANSFORM — clean and
enrich def transform(df: pd.DataFrame) -> pd.DataFrame: df =
df.drop_duplicates(subset=['order_id']) # deduplicate df['revenue'] = df['quantity'] *
df['unit_price'] # derive metric df['order_date'] = pd.to_datetime(df['order_date']) # parse types
df['month'] = df['order_date'].dt.to_period('M').astype(str) df = df[df['revenue'] > 0] # filter
invalid rows logger.info(f'Transformed: {len(df)} valid rows') return df # LOAD — upsert into
warehouse def load(df: pd.DataFrame, table: str) -> None: engine =
sqlalchemy.create_engine('postgresql://user:pass@warehouse/dw') df.to_sql(table, engine,
if_exists='append', index=False, method='multi') logger.info(f'Loaded {len(df)} rows to {table}') #
ORCHESTRATE — run full pipeline def run_pipeline(): last_run = (datetime.now() -
timedelta(hours=1)).isoformat() raw = extract(last_run) clean = transform(raw) load(clean,
'fact_orders')
```

SECTION 3

Data Quality with Great Expectations

```
import great_expectations as gx context = gx.get_context() dataset =
context.sources.pandas_default.read_dataframe(df) # Define expectations (rules your data must
satisfy) dataset.expect_column_to_exist('order_id')
dataset.expect_column_values_to_not_be_null('order_id')
dataset.expect_column_values_to_be_unique('order_id')
dataset.expect_column_values_to_be_between('revenue', min_value=0, max_value=10_000_000)
dataset.expect_column_values_to_be_in_set('status',
['pending','processing','shipped','delivered','cancelled'])
dataset.expect_column_pair_values_A_to_be_greater_than_B( 'revenue', 'cost') # revenue must exceed
cost # Validate — fail fast on quality issues results = dataset.validate() if not
results['success']: failed = [r for r in results['results'] if not r['success']] raise
ValueError(f'Data quality FAILED: {len(failed)} checks failed\n{failed}') logger.info('All data
quality checks passed')
```

SECTION 4

Idempotency and Error Handling

Idempotent Pipelines — Critical for Production

An idempotent pipeline can be run MULTIPLE TIMES with the same result. Why critical: pipelines fail and get retried. Without idempotency, retries create DUPLICATES. Strategies: 1. UPSERT (INSERT ON CONFLICT UPDATE) — update if exists, insert if not 2. Partition overwrite — delete and rewrite entire partition (e.g. date=2024-01-15) 3. Deduplication — always run dedup on target table after load 4. Idempotent key — use unique order_id to prevent double-inserts

```
# Idempotent UPSERT pattern (PostgreSQL) from sqlalchemy import text def upsert_orders(df: pd.DataFrame, engine) -> None: # Stage data in temp table df.to_sql('orders_stage', engine, if_exists='replace', index=False) # UPSERT: update existing, insert new upsert_sql = text(""" INSERT INTO fact_orders (order_id, revenue, status, updated_at) SELECT order_id, revenue, status, updated_at FROM orders_stage ON CONFLICT (order_id) DO UPDATE SET revenue = EXCLUDED.revenue, status = EXCLUDED.status, updated_at = EXCLUDED.updated_at """) with engine.connect() as conn: conn.execute(upsert_sql) conn.commit() # Retry with exponential backoff import time def retry(fn, max_attempts=3, backoff=60): for attempt in range(max_attempts): try: return fn() except Exception as e: if attempt == max_attempts - 1: raise wait = backoff * (2 ** attempt) logger.warning(f'Attempt {attempt+1} failed, retry in {wait}s: {e}') time.sleep(wait)
```

■ PRACTICE Q & A

Q1. What is the difference between ETL and ELT?

A: *ETL: Transform data BEFORE loading — requires separate compute server, rigid schemas. ELT: Load raw first, Transform inside the warehouse using its own compute (SQL/dbt). ELT is modern and preferred with cloud warehouses.*

Q2. What is incremental loading and why is it better than full load?

A: *Extract only rows changed since last run (using updated_at watermark). Much faster and cheaper than full load for large tables. Essential when source systems have millions of rows.*

Q3. What is CDC (Change Data Capture) and when do you need it?

A: *Captures every row-level change (INSERT/UPDATE/DELETE) in near-real-time from the database transaction log (WAL). Use when you need audit trail, real-time replication, or cannot add updated_at column to source.*

Q4. How do you make a pipeline idempotent?

A: *Use UPSERT operations (INSERT ON CONFLICT UPDATE), partition overwrite (rewrite date partition completely), or always run deduplication after load. Safe to retry any number of times without creating duplicates.*

Q5. What is a dead letter queue in data pipelines?

A: *A storage location for records that fail processing after all retries. Failed records go to DLQ for investigation and manual replay, rather than blocking the entire pipeline or silently dropping data.*

■ ETL Pipeline Cheat Sheet	
`pd.read_sql(query, engine)`	Extract from database
`df.to_sql(table, engine)`	Load to database
`if_exists='append'`	Add rows to existing table
`ON CONFLICT (id) DO UPDATE`	Upsert — idempotent insert
`df.drop_duplicates(subset=['id'])`	Deduplicate by key
`updated_at > last_run_ts`	Incremental watermark filter
`expect_column_values_to_not_be_null`	GE null check
`expect_column_values_to_be_unique`	GE uniqueness check

logger.info/warning/error	Always log pipeline steps
Retry with backoff	Retry 3x with exponential wait
Dead Letter Queue	Store failed records for replay
Idempotent	Safe to run multiple times